بسمه تعالی

ساختار دستورهای شرطی

ساختار دستورات شرطی به برنامه این امکان را می دهد که بر اساس برقرار بودن یا نبودن یک شرط، تصمیم‌گیری کند و مسیر اجرای دستورات را تغییر دهد. در این ساختار ابتدا یک شرط بررسی میشود و سپس در صورت درست بودن آن شرط، مجموعه‌ای از دستورات اجرا میگردد.

دستور if در زبان پایتون

دستور if ساده‌ترین و پرکاربردترین ابزار برای پیاده‌سازی ساختار شرطی است. این دستور یک عبارت منطقی را ارزیابی می کند که نتیجه ی آن میتواند True یا False باشد. اگر نتیجه ی شرط True باشد، بلوک کد مربوط به آن اجرا میشود و در غیر این صورت این بلوک کد اجرا نخواهد شد.

نکته: دستورات مربوط به ساختار شرطی(یعنی دستورات زیر if) باید با یک Tab نسبت به خط دستور if جلوتر نوشته شوند. این تورفتگی مشخص میکند که کدام دستورات به شرط مربوطه است. منظور از تورفتگی همان indent میباشد.

انواع دستورات ساختار شرطی if

نوع اول:ساختار شرطی if ساده:شکل کلی استفاده از این دستور به صورت زیر است

```
if condition:
    statement
```

در دستور if منظور از condition همان شرط جلوی if می‌باشد. این شرط یا true است یا false منظور از statement ،دستور یا دستوراتی است که زیر if نوشته می‌شوند و در صورت درست بودن شرط، این دستورات برای ما اجرا می‌شوند. دستوراتی که زیر if نوشته می‌شوند باید با تورفتگی(indentation) همراه باشند. یعنی باید در زیر if یک بار کلید Tab را زده و سپس دستور یا دستورات زیر if را بنویسیم.

نکته:قبل از اینکه مثالی در رابطه با شرط if بیان کنیم لازم است که ابتدا با عملگر های مقایسه ای (Comparison Operators) و همچنین عملگر های منطقی(Logical Operators) آشنا شویم.

الف) عملگر های مقایسه ای (Comparison Operators)

از این عملگرها برای مقایسه دو مقدار یا دو متغیر استفاده میشود و نتیجه ی آنها همیشه یا true میباشد و یا false ،در جدول زیر انواع عملگرهای مقایسه ای معرفی شده است

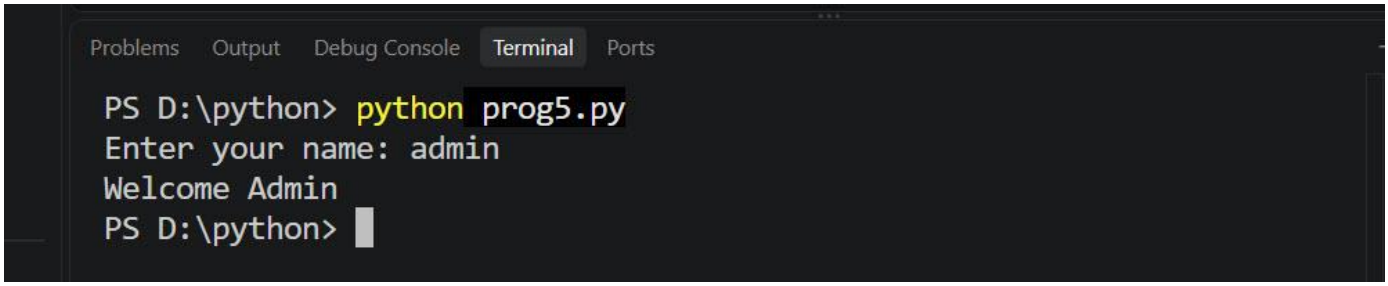
عملگرهای مقایسه‌ای	
عملگر	کاربرد جدول
==	مساوی است
!=	مساوی نیست
>	بزرگ‌تر از
<	کوچک‌تر از
>=	بزرگ‌تر یا مساوی
<=	کوچک‌تر یا مساوی

مثال: برنامه ای به زبان پایتون بنویسید که نام کاربر را دریافت کند. اگر نام وارد شده برابر با مقدار admin باشد، پیام Welcome Admin را برای ما نمایش دهد. برنامه با if ساده نوشته شود.

```
name = input("Enter your name: ")

if name == "admin":
    print("Welcome Admin")
```

خروجی

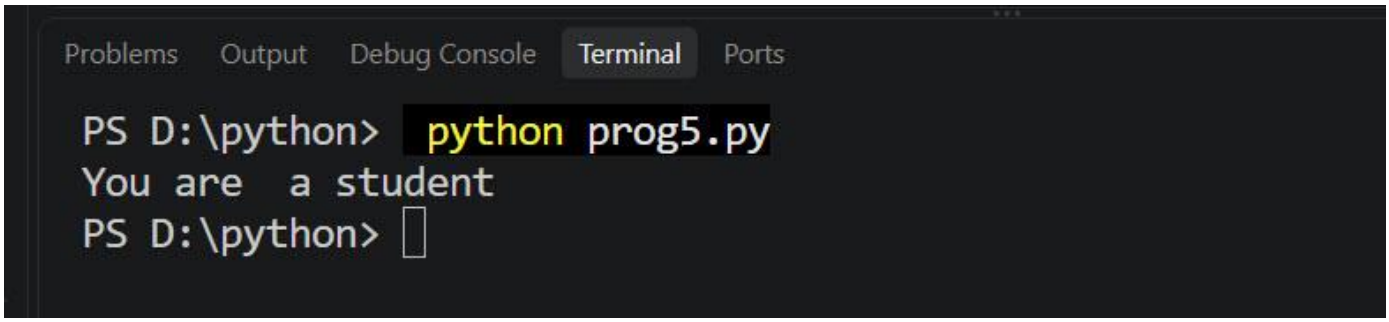


عملکرد عملگر not (!)

در مثال زیر چون is_student برابر Frue است، عبارت not is_student برابر Talse می‌شود و پیام نمایش داده می‌شود.

```
is_student =False

if not is_student:
    print("You are  a student")
```



عملگرهای منطقی یا(Logical Operators)

در برنامه‌نویسی پایتون برای بررسی همزمان چند شرط و تصمیم‌گیری دقیق‌تر از عملگرهای منطقی استفاده می‌کنیم. این عملگرها امکان ترکیب چند شرط را فراهم می‌کنند و نتیجه آنها معمولاً یکی از دو مقدار True یا False می‌باشد. عملگرهای منطقی معمولاً در کنار دستورات شرطی مانند if به کار می‌روند و به برنامه‌نویس کمک می‌کنند شرایط پیچیده‌تری را بررسی کنند. جدول زیر انواع عملگرهای منطقی را نشان می‌دهد.

عملگرهای منطقی	
عملگر	کاربرد
and	و
or	یا
not	نقیض

۱- and — ------ (و) : وقتی استفاده می‌شود که هر دو شرط باید درست باشند.

در مثال زیر چون هر دو شرط درست هستند، پیام نمایش داده می‌شود.

```
age = 20
student = True

if age >= 18 and student:
    print("You are an adult student")
```

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\python> python prog5.py
You are an adult student
PS D:\python>
```

۲- Or ---- (یا) وقتی استفاده می‌شود که حداقل یکی از شرطها درست باشد.

```
age = 16
has_permission = True

if age >= 18 or has_permission:
    print("You can enter")
```

در تکه کد فوق چون شرط دوم درست است، پیام نمایش داده می‌شود

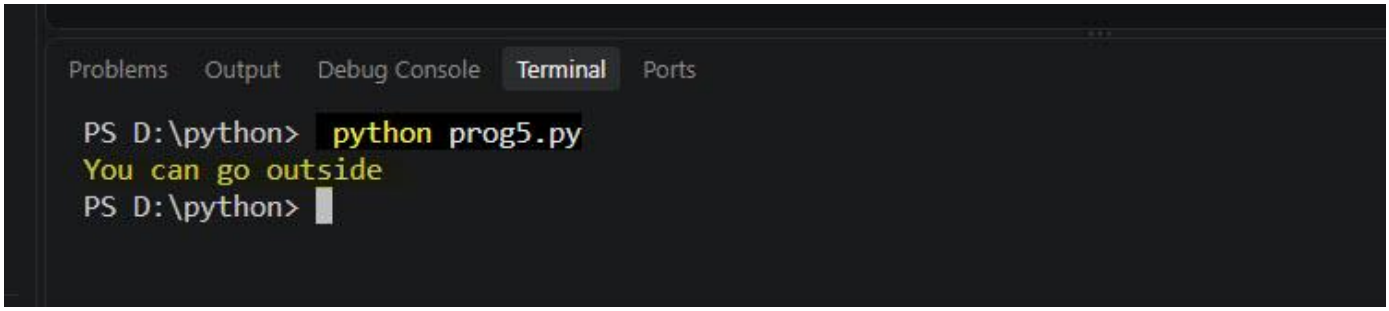


۳- not نقیض / برعکس:نتیجه‌ی یک شرط را برعکس می‌کند؛ True را False و False را True می‌کند.

در تکه کد زیر چون is_raining برابر False است، not is_raining برابر True می‌شود.

```
is_raining = False

if not is_raining:
    print("You can go outside")
```



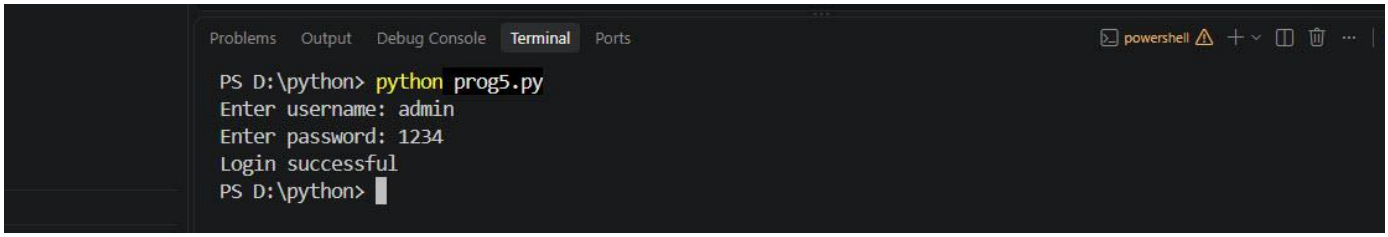
به زبان ساده:

and یعنی هر دو، or یعنی حداقل یکی، و not یعنی برعکس.

مثال:برنامه ای بنویسید که از کاربر یک نام و پسورد ار ورودی گرفته و در صورتیکه نام کاربری admin و پسورد برابر ۱۲۳٤ باشد برنامه پیغام login successfull را نمایش دهد.

```
username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin" and password == "1234":
    print("Login successful")
```



نوع دوم:ساختار شرطی if/else

در این ساختار اگر شرط جلوی **if** درست باشد، دستور یا دستورات زیر **if** که به صورت **indent** یا همان فرورفتگی زیر **if** نوشته می‌شود، برای ما اجرا می‌شود و اگر شرط جلوی **if** غلط باشد، دستور یا دستورات زیر **else** که به صورت **indent** یا همان فرورفتگی زیر **else** نوشته می‌شود برای ما اجرا می‌شود.شکل کلی استفاده از ساختار شرطی **if/else** به صورت زیر است.

```
If condition:
  statements_if
else:
statements_else
```

مثال:برنامه بنویسید که روز هفته را دریافت کرده و در صورتی که جمعه باشد You are off و در غیر این صورت پیام Have to go to school داده شود.

```
day = input("enter day name:")
if day == "friday" or day == "sunday":
    print("you are off!")
else:
    print("you have to go to school!")
```

مثال:برنامه بنویسید که اعداد ما بین صفر و ده را دریافت کند. اگر عدد دریافت شده در این بازه بود پیام The number is valid و در غیر این صورت پیام The number is not valid نمایش داده شود.

```
number = int(input("enter number between 0-10:"))

if number > 0 and number < 10:
    print("The number is valid")
else:
    print("The number is not valid")
```

مثال: برنامه ای بنویسید که نمره درس ریاضی و زبان کاربر را دریافت و اگر هر دو بزرگتر یا مساوی ۱۷ باشد، پیام شما می‌توانید در رشته شبکه و نرم‌افزار ثبت نام کنید را نمایش دهد و در غیر این صورت پیام شما مجاز به انتخاب رشته نیستید را نمایش دهد.

```
math = float(input("Enter your math grade: "))
english = float(input("Enter your English grade: "))

if math >= 17 and english >= 17:
    print("You can enroll in Network and Software.")
else:
    print("You are not eligible to choose this field.")
```

نکته: چون باید هر دو نمره حداقل ۱۷ باشند، از and استفاده می‌کنیم.

مثال: برنامه بنویسید که سن کاربر و وضعیت عضویت او را دریافت کند. اگر سن کاربر بزرگتر یا مساوی ۱۸ باشد و عضو سایت باشد، پیام Access Allowed و در غیر این صورت پیام Access Denied را نمایش دهد.

```
age = int(input("Enter your age: "))
member = input("Are you a member? ")

if age >= 18 and member == "yes":
    print("Access Allowed")
else:
    print("Access Denied")
```

در این مثال هر دو شرط باید درست باشند: سن حداقل ۱۸ سال باشد و کاربر عضو سایت باشد

مثال: برنامه ای بنویسید که کاربر وضعیت اتصال اینترنت و سرعت اینترنت را وارد کند. اگر وضعیت اتصال yes و سرعت بزرگتر یا مساوی سه وارد شد، پیام Internet is Connect چاپ شود و در غیر این صورت پیام Internet is Disconnect چاپ شود.

```
connection = input("Enter connection status: ")
speed = float(input("Enter internet speed: "))

if connection == "yes" and speed >= 3:
    print("Internet is Connect")
else:
    print("Internet is Disconnect")
```

در این برنامه، هر دو شرط باید درست باشند: وضعیت اتصال yes باشد و سرعت اینترنت حداقل ۳ باشد.

مثال: برنامه بنویسید که اگر کاربر مهمان Guest نبود، پیام Welcome نمایش داده شود و در غیر این صورت پیام Access Denied نمایش داده شود.

```
user = input("Enter username: ")

if user != "Guest":
    print("Welcome")
else:
    print("Access Denied")
```

ترکیب دو عملگر AND و OR

در برنامه‌نویسی گاهی نیاز است بیش از یک شرط را با هم بررسی کرد. برای این کار می‌توان عملگرهای منطقی AND و OR را با هم ترکیب کرد.

مثال: برنامه بنویسید که نام کاربری و رمز عبور را دریافت کند و اگر نقش کاربر teacher یا Admin بود و رمز وارد شده برابر با ۱۲۳٤ باشد پیام Access Allowed و در غیر این صورت پیام Access Denied چاپ شود.

نکته:دربرنامه فوق چون نقش کاربر می‌تواند Teacher یا Admin باشد، از or برای نقش‌ها و and برای ترکیب نقش و رمز عبور استفاده می‌کنیم

```
role = input("Enter your role: ")
password = input("Enter your password: ")

if (role == "teacher" or role == "admin") and password == "1234":
    print("Access Allowed")
else:
    print("Access Denied")
```

در مثال فوق:

or یعنی نقش teacher یا admin باشد.

and یعنی علاوه بر نقش صحیح، رمز عبور هم باید ۱۲۳٤ باشد.

مثال: برنامه ای بنویسید که یک کاربر تنها در صورتی اجازه ورود داشته باشد که نام کاربری وارد شده باشد و رمز عبور صحیح باشد یا کد تأیید فعال باشد.

```
username = input("Enter username: ")
password = input("Enter password: ")
verification_code = input("Is verification code active? ")

if (username != "" and password == "1234") or verification_code == "yes":
    print("Access Allowed")
else:
    print("Access Denied")
```

در مثال فوق

and یعنی نام کاربری وارد شده باشد و رمز صحیح باشد.

or یعنی یا کد تأیید فعال باشد.

!="" یعنی نام کاربری خالی نباشد.

مثال: برنامه ای بنویسید که اگر نام کاربری وارد شده نامساوی با guest باشد، از ورود او با پیغام مناسب جلوگیری شود. برنامه را اجرا و نتیجه را با داده های مختلف بررسی کنید.

```
username = input("Enter username: ")

if username != "guest":
    print("Access Denied")
else:
    print("Welcome")
```

در مثال فوق

بررسی با داده‌های مختلف

ورودی کاربر	نتیجه
guest	Welcome
admin	Access Denied
ali	Access Denied
Guest	Access Denied

نکته: پایتون به حروف بزرگ و کوچک حساس است؛ بنابراین guest با Guest متفاوت است.

مثال(عملگر > در بررسی شرط)

برنامه ای بنویسید که نمره ی هنرجو در درس پودمانی را از ورودی گرفته، اگر نمرهی هنرجو بیشتر از دوازده بود، پیام You Passed را نمایش دهد و در غیر این صورت پیام You Failed را نمایش دهد.

```
grade = float(input("Enter your grade: "))

if grade > 12:
    print("You Passed")
else:
    print("You Failed")
```

مثلاً اگر نمره ۱۵ وارد شود، پیام You Passed و اگر نمره ۱۰ وارد شود، پیام You Failed نمایش داده می‌شود.

مثال(عملگر < در بررسی شرط)

برنامه بنویسید که برای ورود به یک سایت سن کاربر را گرفته، اگر سن کاربر کوچکتر از ۱۸ سال بود به او اجازه ورود به سایت داده نشود، در غیر این صورت اجازه داده شود.

```
age = int(input("Enter your age: "))

if age < 18:
    print("Access Denied")
else:
    print("Access Allowed")
```

اگر سن کمتر از ۱۸ باشد، ورود مجاز نیست؛ در غیر این صورت اجازه ورود داده می‌شود.

مثال: برنامه ای بنویسید که شرط ورود به مرحله دوم یک مسابقه را به کاربری بدهد که حداقل سن ۱۸ و امتیاز حداقل ۷۰ را کسب کرده باشد. در غیر این صورت آن کاربر اجازه شرکت در مسابقه را نخواهد داشت.(مثلاً اگر سن ۲۰ و امتیاز ۷۵ باشد، کاربر اجازه ورود به مرحله دوم را دارد.)

```
age = int(input("Enter your age: "))
score = int(input("Enter your score: "))

if age >= 18 and score >= 70:
    print("You can enter the second stage.")
else:
    print("You are not allowed to participate in the competition.")
```

مثال: برنامه ای بنویسید که سن و نام کشور کاربر را دریافت کند و در صورتی که سن بزرگتر یا مساوی ۱۸ باشد و نام کشور ایران بود، پیام اجازه ثبت نام دارید را نمایش دهد.

```
age = int(input("Enter your age: "))
country = input("Enter your country: ")

if age >= 18 and country == "Iran":
    print("You are allowed to register.")
else:
    print("You are not allowed to register.")
```

چون هر دو شرط باید درست باشند، از عملگر and استفاده می‌کنیم

نوع سوم ساختار شرطی If, elif, else

این ساختار زمانی استفاده می‌شود که چند شرط مختلف وجود داشته باشد و برنامه باید یکی از آنها را انتخاب کند. elif شرط جدید را در صورت برقرار نبودن شرط‌های قبلی بررسی می‌کند. یعنی اگر یکی از شرط‌های if یا elifها برقرار شود و دستور آن اجرا شود، دیگر شرط سایر elifها بررسی نخواهد شد. elif مخفف else if می‌باشد و برای بررسی شرط‌های بعدی استفاده می‌شود. تفاوت بین else و elif این است که بعد از else هیچ شرطی نوشته نمی‌شود. ساختار کلی **If, elif, else** به صورت شکل زیر است.

```
if condition1:
    statements1
elif condition2:
    statements2
else:
    statements_else
```

مثال:برنامه ای بنویسید که سطح بازی را از ورودی دریافت و سپس بر اساس آن پیام مناسب نمایش دهد.
برنامه با if-elif-else نوشته شود.

```
level = input("Enter game level: ")

if level == "easy":
    print("Easy level selected")
elif level == "medium":
    print("Medium level selected")
elif level == "hard":
    print("Hard level selected")
else:
    print("Invalid level")
```

در مثال فوق

اگر easy وارد شود ← سطح آسان

اگر medium وارد شود ← سطح متوسط

اگر hard وارد شود ← سطح سخت

در غیر این صورت ← Invalid level

مثال: برنامه ای بنویسید که پس از دریافت نوع غذا و نوشیدنی از کاربر، بر اساس موارد زیر پیام مناسب را چاپ کند: اگر پیتزا و نوشیدنی انتخاب شود، قیمت ۱،۴۰۰،۰۰۰ تومان؛ اگر پیتزا بدون نوشیدنی انتخاب شود، قیمت ۱،۲۰۰،۰۰۰ تومان؛ در غیر این صورت پیام other option چاپ شود.

```
food = input("Enter food: ")
drink = input("Do you want a drink? ")

if food == "pizza" and drink == "yes":
    print("Price: 1,400,000 Toman")
elif food == "pizza" and drink == "no":
    print("Price: 1,200,000 Toman")
else:
    print("Other option")
```

نکته: مزیت استفاده از ساختار if-elif-else نسبت به استفاده از چند if مجزا این است که اگر شرطی برقرار باشد، دیگر شرط های بعدی توسط برنامه بررسی نمی شوند. این ویژگی باعث افزایش سرعت اجرای برنامه و بهینه تر شدن عملکرد آن می شود.

فعالیت

فروشنده برای خریداران میوه سیب تخفیفی گذاشته باشد برنامه بنویسید که نام میوه و تعداد خرید را از کاربر بگیرد. اگر میوه apple و تعداد حداقل ۵ باشد، پیام discount apple applied نمایش داده شود. در غیر این صورت پیام no discount نمایش داده شود.

```
fruit = input("Enter fruit: ")
quantity = int(input("Enter quantity: "))

if fruit == "apple" and quantity >= 5:
    print("Discount apple applied")
else:
    print("No discount")
```

فعالیت

یک برنامه بنویسید که کاربر سن خود را وارد کند. برنامه باید موارد زیر را بررسی کند. اگر سن کاربر بین ۱۸ تا ۲۵ سال باشد، پیام Welcome Young Adult نمایش داده شود. اگر سن کاربر بزرگتر یا برابر ۲۶ باشد، پیام Welcome Adult نمایش داده شود. اگر سن کاربر کمتر از ۱۸ باشد، پیام You are underage نمایش داده شود.

```
age = int(input("Enter your age: "))

if age >= 18 and age <= 25:
    print("Welcome Young Adult")
elif age >= 26:
    print("Welcome Adult")
else:
    print("You are underage")
```

فعالیت

برنامه بنویسید که از دستور if-elif با دریافت وضعیت سفارش از فروشنده استفاده کند و بر اساس وضعیت سفارش که ممکن است Delivered، Sent، Registered باشد، در فروشگاه اینترنتی نحوه ارسال کالا با پیام‌های مناسب اعلام شود.

Delivered:سفارش تحویل داده شده است.

Sent :سفارش ارسال شده است.

Registered سفارش ثبت شده است.

در سایر حالت‌ها : وضعیت سفارش نامعتبر است.

```
status = input("Enter order status: ")

if status == "Delivered":
    print("Your order has been delivered.")
elif status == "Sent":
    print("Your order has been sent.")
elif status == "Registered":
    print("Your order has been registered.")
else:
    print("Unknown order status.")
```

فعالیت

برنامه بنویسید که امتیاز دانش‌آموز را به صورت یک عدد صحیح مابین ۱ تا ۱۰۰ دریافت کند، به نحوی که اگر امتیاز بزرگتر– مساوی ۹۰ باشد پیام Excellent، اگر امتیاز بین ٦۰ تا ۹۰ باشد پیام Good، اگر امتیاز بین ۵۰ تا ٦۹ باشد پیام Pass و اگر امتیاز کمتر از ۵۰ باشد پیام Fail نمایش داده شود.

```
score = int(input("Enter your score (1-100): "))

if score >= 90:
    print("Excellent")
elif score >= 70:
    print("Good")
elif score >= 50:
    print("Pass")
else:
    print("Fail")
```

مثال: برنامه‌ای بنویسید که اگر مقدار choice، یکی از گزینه‌های پیتزا، برگر، پاستا بود، پیام مناسب را چاپ کند و در غیر این صورت پیام Food not available را نمایش دهد.

```
choice = input("Enter your food choice: ")

if choice == "pizza":
    print("You selected Pizza")
elif choice == "burger":
    print("You selected Burger")
elif choice == "pasta":
    print("You selected Pasta")
else:
    print("Food not available")
```

نکته: در مثال بالا اگر منوی غذا بیشتر از ۲۰غذا باشد، چه باید کرد؟ یا اگر فردا غذای جدیدی اضافه شود، چه باید کرد؟ اولین راه‌حلی که به نظر می‌رسد این است که از تعداد بیشتری if استفاده شود، ولی این کار کد را طولانی‌تر و سخت‌تر می‌کند. بنابراین راه‌حل مناسب، استفاده درست از لیست‌ها و همچنین استفاده از دستورات حلقه‌های تکرار می‌باشد که در مبحث بعدی مورد بررسی قرار خواهند گرفت.

حلقه‌های تکرار(loop)

در هنگام کدنویسی، اگر نیاز باشد یک دستور یا مجموعه‌ای از دستورات چندین بار اجرا شود، از ساختارهای حلقه تکرار استفاده می‌شود. این ساختارها باعث می‌شوند کدها خواناتر، کوتاه‌تر و مدیریت آن آسان‌تر باشند. به عنوان مثال، اگر لازم باشد برنامه ای نوشته شود که کلمه پایتون را ده بار چاپ کند، به جای آنکه ده بار از دستور print به طور جداگانه استفاده کنیم، می‌توانیم این دستور را تنها یک بار در داخل حلقه قرار داده تا با اجرای آن ،حلقه به صورت خودکار ده بار برای ما تکرار شود.

در زبان برنامه‌نویسی پایتون برای پیاده‌سازی حلقه‌های تکرار دو ساختار اصلی وجود دارد: ۱–حلقه for، ۲– حلقه while

الف)حلقه for

برای تکرار اجرای دستورات روی یک دنباله مانند لیست، رشته، تاپل، مجموعه و دیکشنری، می‌توانیم از حلقه for استفاده کنیم و معمولاً زمانی از این حلقه استفاده می‌شود که تعداد تکرارها از قبل مشخص یا قابل تعیین باشد. ساختار کلی حلقه for در زبان پایتون به شکل زیر می‌باشد.

for variable in sequence:

statements

در ساختار فوق، منظور از variable متغیری است که در هر تکرار مقدار یکی از عناصر دنباله را دریافت می‌کند. نام متغیر می‌تواند دلخواه باشد، مانند i، count، num، student و غیره. منظور از sequence ،یک دنباله مانند list، رشته، tuple، مجموعه یا دیکشنری است. منظور از statement مجموعه دستوراتی در بدن حلقه هستند که در هر بار تکرار اجرا می‌شوند. به این دستورات اصطلاحاً بلاک حلقه گفته می‌شود و بایستی به اندازه یک تب، تورفتگی یا indent داشته باشند.

مثال(پیمایش لیست با حلقه for) تکه کدی به زبان پایتون بنویسید که عناصر یک لیست شامل سه رنگ green، blue و redرا به کمک حلقه for چاپ کند.

```
colors = ["Green", "Blue", "Red"]

for color in colors:
    print(color)
```

مثال:

در مثال زیر سبد خریدی که با لیستی با نام Shopping Cart ایجاد شده است با کالاهایی در آن ذخیره شده است، سپس عناصر موجود در لیست با استفاده از حلقه for چاپ میشود.

```
shopping_cart = ["Ruby", "Milk", "Cheese", "Cream", "Honey", "Bread"]

for item in shopping_cart:
    print("color:" , item)
```

مثال: چاپ حروف کلمه پایتون با حلقه for

```
word = "Python"

for letter in word:
    print(letter)
```

نمونه هایی از کاربرد حلقه for

لیست زیر شامل گزینه‌هایی می‌باشد. برنامه‌ای با زبان پایتون و به کمک حلقه for بنویسید که تعداد عناصر این لیست را به ما نشان دهد.

```
fruits = ["apple", "banana", "apple", "orange", "apple"]

count = 0

for fruit in fruits:
    count += 1

print("Number of elements:", count)
```

در مثال فوق هر بار که حلقه for اجرا می شود یک عنصر را بررسی می‌کند و مقدار count یک واحد افزایش پیدا می‌کند

مثال: در مثال بالا اگر بخواهیم تعداد یک عنصر خاص را از لیست به عنوان مثال تعداد apple را مشخص کنیم، می‌توانیم از دستور if در درون حلقه به صورت زیر استفاده کنیم.

```

fruits = ["apple", "banana", "apple", "orange", "apple"]

count = 0

for fruit in fruits:
    if fruit == "apple":
        count += 1

print("Number of apples:", count)
```

نکته:به کمک متد count() می توان کار برنامه فوق را راحت تر انجام داد.

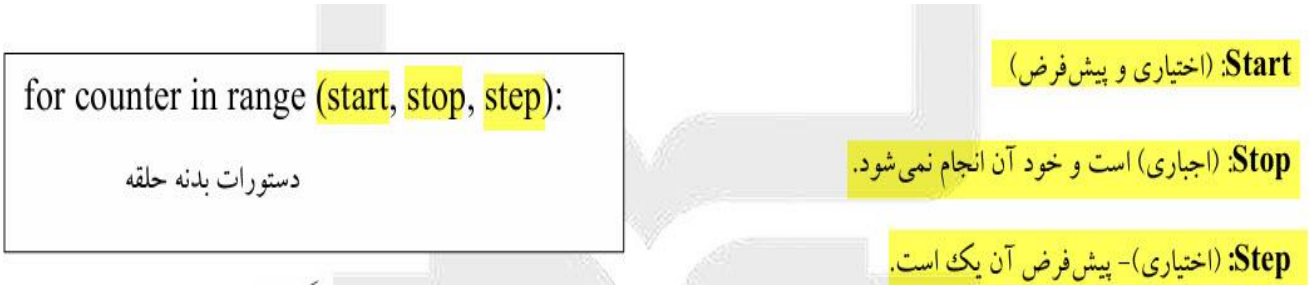
```
fruits = ["apple", "banana", "apple", "orange", "apple"]

print("Number of apples:", fruits.count("apple"))
```

بنابراین،متد count()تعداد تکرار یک عنصر مشخص در لیست را برمی‌گرداند.

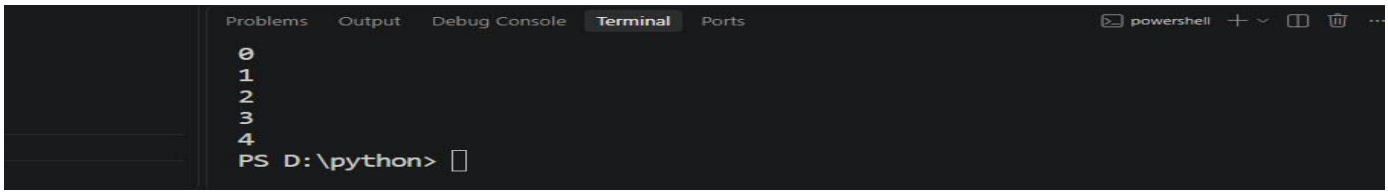
تابع range()

تا اینجای کار از حلقه For برای پیمایش عناصر موجود در لیست‌ها و رشته‌ها استفاده شده است. در برخی موارد لازم است یک دستور به تعداد دفعات مشخصی تکرار شود، بدون آنکه از قبل لیستی از مقادیر در اختیار باشد. در چنین شرایطی از تابع Range استفاده می‌شود. تابع Range یک دنباله از اعداد تولید می‌کند که از مقدار Start آغاز شده و با گام‌هایی که در Step مشخص می‌شود تا قبل از مقدار Stop ادامه می‌یابد. شکل کلی استفاده از این تابع به صورت زیر می‌باشد.



در ساده‌ترین حالت، تابع range()فقط یک عدد دریافت می‌کند که به عنوان stop در نظر گرفته می‌شود.

```
for i in range(5):
    print(i)
```



نکته:در مثال فوق عدد ۵ در range(5) شامل خروجی نمی‌شود؛ شمارش از ۰ شروع می‌شود و تا ٤ ادامه پیدا می‌کند.

حالت دوم استفاده از تابع range():در این حالت تابع range() دو عدد دریافت می‌کند: start و stop

مثال، حلقه ای برای چاپ اعداد ۱ تا ٦ بنویسید.

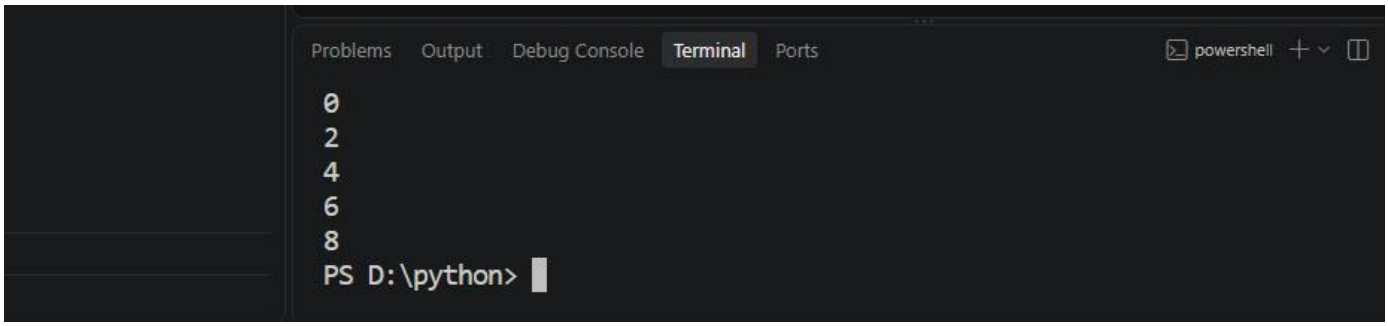
```
for i in range(1, 7):
    print(i)
```

نکته: عدد stop در range()چاپ نمی‌شود؛ بنابراین برای اینکه عدد ٦ چاپ شود، مقدار stop را ۷ قرار می‌دهیم.

حالت سوم استفاده از تابع range() : در حالت سوم، تابع range()سه مقدار دریافت می‌کند

range(start, stop, step)

```
for i in range(0, 10, 2):
    print(i)
```



در اینجا:

۰ ⟵ مقدار start

۱۰ ⟵ مقدار stopخود ۱۰ چاپ نمی‌شود

۲ ⟵ مقدار step؛ یعنی هر بار ۲ واحد جلو می‌رویم.

فعالیت: مثال فوق را به گونه ای تغییر دهید که دنباله ای اعداد زوج را از بزرگ به کوچک چاپ کند. برای منظور لازم است مقدار step منفی شده و مقادیر شروع و پایان نیز متناسب با آن تغییر داده شود.

```
for i in range(8, 0, -2):
    print(i)
```

```
PS D:\python> python prog5.py
8
6
4
2
PS D:\python>
```

در اینجا:

۸ ⟵ مقدار start

۰ ⟵ مقدار stop (خود ۰ چاپ نمی‌شود)

-2 ⟵ مقدار step؛ یعنی هر بار ۲ واحد به عقب می‌رویم.

فعالیت:

برنامه ای بنویسید که مدت زمان انتظار برای دریافت غذا را به صورت یک عدد برحسب دقیقه از کاربر دریافت کرده و شمارش معکوس را نمایش دهد. در هر مرحله پیام زیر چاپ شود: Time remaining: X minutes، که در آن به جای X مقدار زمان باقی مانده نمایش داده می‌شود. پس از رسیدن مقدار شمارش به صفر پیام زیر نمایش داده شود: Your food is ready

در برنامه زیر برای چاپ محتوای هر بار مقدار i ،در دستور print از f” با فرمتی که نشان داده ایم استفاده شده است.

```
minutes = int(input("Enter waiting time in minutes: "))

for i in range(minutes, 0, -1):
    print(f"Time remaining: {i} minutes")

print("Your food is ready.")
```

فعالیت

برنامه بنویسید که قد پنج نفر را از ورودی گرفته و میانگین قد آنها را محاسبه و نمایش دهد.

برای این کار می‌توانیم قد هر ۵ نفر را با حلقه for دریافت کنیم، سپس مجموع را بر ۵ تقسیم کنیم.

```
total = 0

for i in range(5):
    height = float(input("Enter height: "))
    total += height

average = total / 5

print("Average height:", average)
```

در این برنامه، متغیر total مجموع قدها را نگه می‌دارد و در پایان، میانگین با تقسیم مجموع بر ۵ محاسبه می‌شود.

نکته: در حلقه for می‌توان از کاراکتر زیرخط یا underline در نام متغیرها استفاده کرد. استفاده از underline به جای استفاده از نام متغیر چه مزیتی دارد؟

در پایتون وقتی از _ به‌عنوان متغیر حلقه استفاده می‌کنیم، معمولاً یعنی مقدار هر تکرار برای ما مهم نیست و فقط می‌خواهیم حلقه به تعداد مشخصی اجرا شود.

```
for _ in range(5):
    print("Hello")
```

```
Problems  Output  Debug Console  Terminal  Ports
PS D:\python> python prog5.py
Hello
Hello
Hello
Hello
Hello
```

اگر به جای _ از i استفاده کنیم(خروجی تکه کد زیر ۰ تا ٤ خواهد بود.)

```
for i in range(5):
    print(i)
```

پس مزیت _ این است که به خواننده‌ی برنامه نشان می‌دهد ما به مقدار متغیر حلقه نیازی نداریم و فقط تعداد دفعات اجرای حلقه برایمان مهم است.

معرفی دستورات کنترلی break و continue در حلقه‌ها:

در زبان پایتون، دستورات کنترلی break و continue برای کنترل جریان اجرای حلقه‌ها استفاده می‌شوند. از continue برای رد شدن از یک بخش خاص از حلقه و رفتن به تکرار بعدی استفاده می‌شود و break باعث می‌شود اجرای حلقه به طور کامل متوقف شده و از آن خارج شود.

مثال break

وقتی به عدد ۳ می‌رسد، حلقه کاملاً متوقف می‌شود:

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

مثال continue

وقتی به عدد ۳ می‌رسد، فقط همان دور را رد می‌کند و حلقه ادامه پیدا می‌کند:

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

خلاصه:

break :کل حلقه را متوقف می‌کند.

continue : فقط همان مرحله را رد می‌کند و به حلقه ادامه می‌دهد.

حلقه while()

این حلقه مجموعه‌ای از دستورات را تا زمانی که شرط حلقه **true** باشد اجرا می‌کند و هر زمان که شرط حلقه **false** شود دیگر حلقه اجرا نمی‌شود و اجرای حلقه پایان می‌یابد. از این نوع حلقه معمولاً زمانی استفاده می‌شود که تعداد دفعات تکرار حلقه از قبل مشخص و معین نباشد. شکل کلی استفاده از حلقه **while** به صورت زیر می‌باشد.

شرط حلقه While

دستورات بدنه حلقه

مثال: شخصی در ابتدای یک تالار غذاخوری به مهمانان یک مراسم عروسی خوشامد می‌گوید. به او گفته شده که آنجا بایستد و بعد از پرسیدن نام آنها به مهمان وارد شده خوشامد بگوید و برای پایان کار او محدودیتی در نظر گرفته نشده است. برنامه را به کمک حلقه **while** بنویسید.

```
while True:
    name = input("Please enter your name: ")

    if name == "":
        break

    print(f"Welcome, {name}!")
```

در مثال فوق **while True** باعث می‌شود حلقه به‌صورت مداوم اجرا شود.

با **input** نام مهمان دریافت می‌شود.

اگر کاربر نامی وارد نکند، شرط **name == ""** برقرار شده و با **break** حلقه تمام می‌شود.

اگر نام وارد شود، برنامه با استفاده از **print** به مهمان خوشامد می‌گوید.

مثال: برنامه بالا را طوری بنویسید که بعد از ورود داماد وظیفه خوشامدگویی پایان یابد.

```
while True:
    name = input("Please enter your name: ")

    if name == "Damad":
        print("Welcome, Damad!")
        break

    print(f"Welcome, {name}!")
```

در مثال فوق حلقه while تا زمانی که داماد وارد نشده است ادامه پیدا می‌کند.

اگر نام واردشده Damad باشد، برنامه به او خوشامد می‌گوید.

سپس دستور break اجرا شده و وظیفه‌ی خوشامدگویی پایان می‌یابد.

برای سایر مهمانان، برنامه همچنان به خوشامدگویی ادامه می‌دهد.

مثال:

در مثال فوق، به جز مهمانان، کارمندان نیز در حال رفت و آمدند. حال اگر نخواهیم به کارمندان خوشامد گفته شود، به کمک یک دستور شرطی و دستور continue برنامه را بنویسید.

```
while True:
    name = input("Please enter your name: ")

    if name == "Employee":
        continue

    if name == "Damad":
        print("Welcome, Damad!")
        break

    print(f"Welcome, {name}!")
```

گر فرد واردشده Employee باشد، شرط اول برقرار می‌شود.

دستور continue باعث می‌شود برنامه بدون خوشامدگویی، مستقیماً به ابتدای حلقه برگردد.

اگر فرد Damad باشد، به او خوشامد گفته می‌شود و با break حلقه پایان می‌یابد.

برای سایر افراد، پیام خوشامدگویی نمایش داده می‌شود.

نکته: در برنامه فوق، اگرچه برنامه به خوبی کار می‌کند، اما این ساختار به خاطر وجود true در جلوی while برای برنامه‌نویسی خیلی مناسب نیست و می‌توان ساختار حلقه را به صورت مثال زیر تغییر داد. در مثال زیر به مسئول خوشامدگویی گفته شود که این کار را تا زمانی ادامه دهد که داماد نیامده است.

```
name = ""

while name != "Damad":
    name = input("Please enter your name: ")

    if name == "Employee":
        continue

    print(f"Welcome, {name}!")
```

در مثال فوق

ابتدا متغیر name را خالی قرار می‌دهیم.

شرط while name != "Damad" یعنی تا زمانی که داماد وارد نشده است، حلقه ادامه داشته باشد.

اگر فرد واردشده کارمند باشد، continue باعث می‌شود بدون خوشامدگویی، حلقه از ابتدا ادامه پیدا کند.

وقتی نام Damad وارد شود، شرط حلقه نادرست می‌شود و برنامه پایان می‌یابد.

مثال:

در برنامه فوق به مسئول خوشامدگویی گفته شده که آنجا بایستد و بعد از پرسیدن نام آنها به مهمان وارد شده خوشامد بگوید و به محض ورود داماد وظیفه خوشامدگویی پایان می‌پذیرد. اما دیگر نیازی نیست به داماد خوشامد گفته شود. با این شرایط برنامه را بنویسید.

```
name = ""

while name != "Damad":
    name = input("Please enter your name: ")

    if name == "Damad":
        break

    if name == "Employee":
        continue

    print(f"Welcome, {name}!")
```

حلقه تا زمانی ادامه دارد که داماد وارد نشده باشد.

اگر Damad وارد شود، با break حلقه بلافاصله پایان می‌یابد؛ بنابراین به داماد خوشامد گفته نمی‌شود.

اگر Employee وارد شود، دستور continue باعث می‌شود بدون خوشامدگویی به ابتدای حلقه برگردیم.

برای سایر افراد، پیام خوشامدگویی نمایش داده می‌شود.

مثال:برنامه ای بنویسید که از کاربر یک عدد صحیح دریافت و مربع آن عدد را چاپ کند. این کار تا زمانی ادامه یابد که کاربر عددی بزرگتر از صفر وارد کند.

```
number = int(input("Enter an integer: "))

while number > 0:
    print("Square:", number ** 2)
    number = int(input("Enter an integer: "))
```

در مثال فوق اگر کاربر ۳، سپس ۵ و بعد –۱ وارد کند، خروجی: مربع عدد۳: ۹ مربع عدد۵: ۲۵ ،با وارد کردن ۱– برنامه متوقف می‌شود

مثال: تکه کدی بنویسید که اعداد ۱ تا ۵ را به کمک حلقه for زیر هم چاپ کند و همین برنامه را یک بار دیگر بنویسید که به کمک حلقه while اعداد ۱ تا ۵ را زیر هم چاپ کند.

```
for i in range(1, 6):
    print(i)
```

هر دو برنامه اعداد ۱ تا ۵ را زیر هم چاپ می‌کنند.

```
i = 1

while i <= 5:
    print(i)
    i += 1
```

مثال: برنامه ای بنویسید که نام تعدادی غذا را از کاربر گرفته و به یک لیست خالی اضافه یا append کند. این کار تا زمانی ادامه میابد که کاربر کلمه exit را وارد کند. پس از اتمام حلقه محتوای لیست به صورت یکجا چاپ خواهد شد.

```
foods = []

food = input("Enter a food: ")

while food != "exit":
    foods.append(food)
    food = input("Enter a food: ")

print(foods)
```

در برنامه فوق و در انتهای برنامه بالا به کمک حلقه for تک تک غذاهای لیست را زیر هم چاپ کنید.

پاسخ: فقط در انتهای برنامه، یک حلقه for اضافه می‌کنیم تا غذاها را یکی‌یکی زیر هم چاپ کند:

```
foods = []

food = input("Enter a food: ")

while food != "exit":
    foods.append(food)
    food = input("Enter a food: ")

for food in foods:
    print(food)
```

خروجی

Pizza

Burger

Pasta

فعالیت:

برنامه ای را بنویسید که کاربر بتواند به صورت مکرر سفارش غذا دهد تا زمانی که Done را وارد کند. هر بار که غذای جدیدی از کاربر گرفته شد، پیام Your order for food has been added نمایش داده شود و غذای جدید به لیست سفارش غذا اضافه یا Append شود. وقتی کاربر Done را وارد نمود، برنامه تمام سفارش‌ها را نمایش داد.

برنامه با حلقه while، لیست خالی و append به این صورت است:

```
orders = []

food = input("Enter your order: ")

while food != "Done":
    orders.append(food)
    print("Your order for", food, "has been added")
    food = input("Enter your order: ")

print("Your orders:")

for food in orders:
    print(food)
```

مثلاً اگر کاربر وارد کند:

Pizza

Burger

Pasta

Done

خروجی به صورت زیر خواهد بود.

Your order for Pizza has been added

Your order for Burger has been added

Your order for Pasta has been added

Your orders:

Pizza

Burger

Pasta